

PintOS - Relatório de Implementação

Projeto 2 - User Programs

OBS: Embora a passagem de argumentos esteja 100% implementada, não consegui concluir a syscall `exit()` e, portanto, nenhum teste desta seção em diante está funcionando. Este relatório reflete mais o meu entendimento exato do que precisa ser implementado no projeto do que o resultado dos meus esforços (individuais) de implementar, pois tive muitas dificuldades com o código.

Parte 1 - Carregamento de Processos e Passagem de Argumentos

`userprog/process.c`, `userprog/process.h`

- Implementado tokenizador de linha de comando `tokenize_arguments` com `strtok_r()`
- Modificados `process_execute()` e `start_process()` para passar os argumentos separadamente
- Modificado `setup_stack()` para posicionar argumentos na pilha

Design

1. Thread pai chama `process_execute(exec_string)` :
 1. Cópia de `exec_string`
 2. Tokenização com `strtok_r()` na forma de `argc` e `**argv`
 3. Invoca thread `start_process()` de nome `argv[0]`.
2. Em `start_process()`, carrega o executável via `load()`
3. Cria `intr_frame` e configura a pilha de usuário:
 1. Ajuste inicial de `esp` para `PHYS_BASE`.
 2. Cópia inversa dos argumentos para a pilha.
 3. Alinhamento de stack a múltiplos de 4 bytes.
 4. Empilhamento de ponteiros `argv[]`, `argc` e endereço de retorno falso.

Resultados de Testes

- `args-none`: Sem argumentos.
- `args-single`: Um argumento.
- `args-multiple`: Múltiplos argumentos.
- `args-dbl-space`: Espaços duplos tratados corretamente.

Parte 2 - Controle de Processos (`halt`, `exit`, `exec`, `wait`)

`userprog/process.c`, `userprog/syscall.c`, `threads/thread.c`

- Implementadas funções de syscall: `halt()`, `exit(status)`, `exec(cmd)`, `wait(pid)`
- Modificada `syscall_handler()` para despachar chamadas de sistema e extrair argumentos da pilha do usuário
- Adicionada impressão do status de saída em `exit()`
- Implementada sincronização entre processos pai e filho usando semáforos e variáveis de condição
- Estruturado gerenciamento de status de saída e comunicação entre threads

Design

1. O processo de usuário faz uma chamada de sistema (ex: `exit`, `exec`, `wait`, `halt`)
2. O handler de syscall (`syscall_handler`) identifica o código da syscall e extrai os argumentos da pilha do usuário
3. Para `exec`, uma nova thread de usuário é criada, inicializando estruturas de controle de filho e retornando o `tid`
4. Para `wait`, o processo pai bloqueia até que o filho termine, usando semáforo/condição para sincronização
5. Para `exit`, o processo registra o status de saída, imprime a mensagem e libera recursos, sinalizando o pai se necessário
6. Em caso de erro de validação de ponteiro ou falha de carregamento, a thread termina com `exit(-1)`

Resultados de Testes

- `halt.ck`: Desliga o sistema.
- `exit-basic`: Exit básico imprime status.
- `exec-multiple`: Execução de múltiplos programas.
- `wait-single` / `wait-multiple`: Espera correta de filhos.

Parte 3 - Interface Geral de Syscalls e Validação de Ponteiros

`userprog/syscall.c`, `lib/kernel/**/*.c`

- Implementadas funções auxiliares para validação de ponteiros de usuário (`get_user()`, `verify_user_address()`)
- Modificada `syscall_handler()` para validar todos os ponteiros antes de acessar memória do usuário
- Adicionado tratamento de erro para page faults e acessos inválidos, abortando a thread de forma limpa
- Centralizado o despacho de syscalls em uma tabela de funções

Design

1. Ao receber uma syscall, os argumentos são empacotados a partir de `f->esp` em um array de inteiros
2. Antes de acessar qualquer argumento ou ponteiro fornecido pelo usuário, chama-se `verify_user_address()` para garantir que o endereço é válido
3. Se for detectado um ponteiro inválido, a thread termina imediatamente com `exit(-1)`
4. O dispatcher de syscalls utiliza uma tabela para mapear códigos de syscall para funções específicas
5. Todos os acessos a buffers de leitura/escrita passam por validação antes de serem utilizados

Resultados de Testes

- `bad-read`: Syscall com ponteiro de leitura inválido aborta.
- `bad-write`: Escrever em área inválida retorna erro.
- `boundary-*.ck`: Limites de pilha e data testados.

Parte 4 - Chamadas de Sistema de Arquivo

`userprog/syscall.c`, `filesys/file.c`, `filesys/inode.c`

- Implementadas syscalls de arquivos: `create()`, `remove()`, `open()`, `filesize()`, `read()`, `write()`, `seek()`, `tell()`, `close()`
- Modificada a estrutura de cada thread para manter uma tabela de file descriptors (`struct file_descriptor`)
- Adicionada sincronização de acesso ao sistema de arquivos usando locks globais
- Delegadas operações de arquivos para funções do subsistema `filesys`

Design

1. Ao chamar `open()`, o arquivo é aberto e um novo file descriptor único é atribuído à thread, ou retorna -1 em caso de erro
2. As syscalls `read()` e `write()` validam os buffers de usuário e delegam a leitura/escrita para `file_read()` e `file_write()`
3. `seek()` e `tell()` manipulam a posição de leitura/escrita do arquivo associado ao file descriptor
4. `close()` remove o file descriptor da tabela da thread e fecha o arquivo correspondente
5. Todas as operações de arquivos são protegidas por um lock global para garantir acesso concorrente seguro

Resultados de Testes

- `create-basic`: Cria e remove arquivos.
- `open-basic`: Abertura de arquivos e fd válido.
- `read-write`: Leitura e escrita funcionais.
- `seek-tell`: Posição de leitura correta.
- `close-basic`: Fecha descriptor limpo.

Parte 5 - Compartilhamento de Descritores e Processos Filhos

`userprog/syscall.c`, `threads/thread.c`, `filesys/file.c`

- Implementada herança de file descriptors durante `exec()` para processos filhos
- Adicionada sincronização de acesso concorrente a arquivos com lock global do sistema de arquivos
- Estruturada lista de gerenciamento de filhos (`child_info`) em cada thread para controle de status e sincronização

Design

1. Ao executar `exec`, a tabela de file descriptors do processo pai é duplicada para o processo filho, permitindo herança de arquivos abertos
2. Todas as operações de leitura/escrita/fechamento de arquivos utilizam o lock global para evitar condições de corrida
3. Cada thread mantém uma lista de filhos (`child_info`) com status de término e semáforo para sincronização
4. O pai pode chamar `wait()` para aguardar o término de um filho específico, liberando o registro após o término
5. O gerenciamento de recursos garante que file descriptors e estruturas de filhos sejam liberados corretamente ao final do processo

Resultados de Testes

- `multi-child-fd`: Filho herda e compartilha fds.
- `multi-recurse`: Execuções aninhadas de processos.
- `rox-edge`: Acesso simultâneo a arquivos síncrono.